

5 15/06/2021

5.1 Traccia da 12 punti

5.1.1 2pt

Sia data una matrice quadrata di interi M (di dimensione $N \times N$). Ogni cella della matrice può contenere solo il valore 0 o il valore 1. La matrice rappresenta delle relazioni di amicizia tra persone. Se in posizione i,j è contenuto il valore 1, le persone i e j sono amiche. Scrivere una funzione che individui e stampi tutte le coppie aventi esattamente k amici in comune

Esempio

$$M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Coppie valide:

(0, 4) (1, 2) (1, 3) (1, 4) (1, 5) (3, 4) (3, 5) (4, 5)

5.1.2 4pt

Definire una struttura dati adeguata a rappresentare una lista singolo linkata di stringhe come ADT I classe, dove il tipo lista si chiami `LIST`. Indicare esplicitamente in quale modulo/file appare la definizione dei tipi proposti. La struttura così definita è quella da usare per completare la seconda domanda di questo esercizio. **Non è ammesso l'uso di funzioni di libreria.** Implementare una funzione caratterizzata dal seguente prototipo:

```
void split(LIST l, char *str, char sep)
```

la funzione `split` suddivide la stringa `str` in una lista di (sotto)stringhe spezzando `str` ogni volta che viene incontrato il carattere `sep`. Il carattere `sep` deve essere incluso come primo carattere della (sotto)stringa successiva a quella corrente **Attenersi al prototipo dato.**

Esempio 1

`s = "abracadabra" sep = 'r'`
ab → *racadab* → *ra*

Esempio 2

`s = "ramarro" sep = 'r'`
rama → *r* → *ro*

5.1.3 6pt

Data una funzione wrapper con il seguente prototipo

```
void solve(char *start, char *end, sub *S, int nSubs);
```

Dove `start` e `end` sono due generiche stringhe lunghe uguali. `sub` è una struttura definita come segue

```
typedef struct {  
    char *s;  
    int pos;  
    int costo;  
} sub;
```

usata per rappresentare una sostituzione. Una sostituzione è caratterizzata da una sottostringa `s`, con cui sovrascrivere una porzione di una stringa destinazione a partire dalla posizione `pos`. Ogni sostituzione ha un certo costo associato.

Esempio di sostituzione

`start = "amico" s1 = {s = "BA", pos = 1, cost = 4}`
applicando la sostituzione `s1` alla stringa `start`, si ottiene `"aBAco"`.
`s2 = {s = "TE", pos = 3, cost = 1}`
applicando una ulteriore sostituzione `s2`, si ottiene `"aBATE"`.
La trasformazione ha complessivamente costo $4 + 1 = 5$.

`S` è un vettore di sostituzioni ammissibili, di lunghezza `nSubs`. Si assuma per semplicità che tutte le sostituzioni presenti in `S` siano totalmente applicabili alla stringa `start` ricevute in input, ossia che la posizione di applicazione e la lunghezza delle sostituzioni non vada a superare la lunghezza della stringa destinazione. Scrivere una funzione ricorsiva che determini se è possibile individuare una sequenza di sostituzioni, tra quelle disponibili nel vettore `S`, in grado di trasformare la stringa `start`

nella stringa **end**, minimizzando il costo totale di trasformazione. A parità di costo prediligere sequenze di sostituzioni più lunghe.

```
void solve(char *start, char *end, sub *S, int nSubs)
```

Giustificare la scelta del modello combinatorio adottato. Giustificare la scelta del/dei criteri di pruning adottati, o il motivo della loro assenza.

Esempio di completo

```
start = "passato"
end = "persona"
nSubs = 8
S = {
    {"er", 1, 4},
    {"ers", 1, 5},
    {"sa", 3, 1},
    {"so", 3, 2},
    {"ato", 0, 1},
    {"on", 4, 2},
    {"ona", 4, 3},
    {"a", 6, 1}
}
```

```
passato
+ er in posizione 1 a costo 4 = persato
+ on in posizione 4 a costo 2 = persono
+ a in posizione 6 a costo 1 = persona
Costo complessivo di trasformazione: 7
```

5.2 Traccia da 18 punti

5.2.1 Breve introduzione e definizioni

Un magazzino di stoccaggio è costituito da C corridoi. Ogni corridoio ospita C_s scaffali e ogni scaffale ha a disposizione un numero complessivo fisso K di slot, numerati da 0 a $K-1$. Ogni corridoio è caratterizzato da un identificativo numerico unico, da 0 a $C-1$, e dal numero di scaffali che ospita. Ogni corridoio può ospitare un numero diverso di scaffali. Ogni scaffale è caratterizzato da un codice identificativo alfanumerico unico all'interno del magazzino. Si assuma che tutti gli scaffali del magazzino abbiano il medesimo numero di slot. Il magazzino è utilizzato per conservare dei pacchi. Ogni pacco è caratterizzato da un codice alfanumerico unico.

Per ogni pacco si è interessati a memorizzare la posizione dello stesso nel magazzino, in termini della terna `<corridoio, scaffale, slot>`. Si assuma che tutti i pacchi siano di egual dimensione, e occupino un singolo slot ciascuno.

5.2.2 Strutture dati

Definire delle opportune strutture dati così da poter rappresentare lo scenario precedentemente descritto, rispettando i seguenti vincoli:

- tipo **Pacco**: implementazione a discrezione del candidato
- tipo **Scaffale**: implementazione a discrezione del candidato
- tipo **Corridoio**: collezione di **Scaffale** (ADT I categoria)
- tipo **Magazzino**: collezione di **Corridoio** (ADT I categoria)

La base dati deve supportare le seguenti operazioni:

- Ricerca della posizione di un pacco, dato il codice, **con complessità al più logaritmica nel numero dei pacchi totali del magazzino**. Per questo punto, in caso si voglia usare un *BST*, si assuma che siano rispettate automaticamente le proprietà di bilanciamento. In caso si faccia uso di strutture dati di libreria, indicare esplicitamente la tipologia di “*Item*” contenuto e l’eventuale definizione del nodo contenitore;
- Inserimento di un pacco in una data posizione, con eventuale indicazione di errore a fronte di posizione già occupata/non esistente;
- Estrazione di un pacco da una data posizione, con eventuale indicazione di errore a fronte di posizione vuota/non esistente;
- Spostamento di un pacco da una posizione a un’altra, eventualmente in un corridoio/scaffale diverso, con gestione degli errori di cui sopra;
- Compattazione dei contenuti di due scaffali in uno solo dei due, se possibile

Organizzare la soluzione suddividendo il codice in più moduli, uno per ogni tipo richiesto, e un client principale.